

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación

Laboratorio 3.1

Sistema de Parquadero – Microservicio Audit

Autor: Cáceres Villagómez Germán Patricio

NRC: 30731 – Aplicaciones Distribuidas

Docente: Ing. Ángel Cudco, Mgs.

Fecha: 19 de julio de 2026

Sangolquí – Ecuador

Índice

Resumen	2
1. Introducción	3
2. Objetivos	3
Objetivo General	3
Objetivos Específicos	4
3. Desarrollo	5
3.1. Arquitectura general del sistema	5
3.2. Patrón productor-consumidor con RabbitMQ	5
3.3. Esquema de datos y validación	7
3.4. Seguridad: autenticación y autorización	7
3.5. Configuración de despliegue y enrutamiento	8
3.6. Verificación funcional	9
4. Conclusiones	10
5. Recomendaciones	11
Bibliografía	12

Resumen

El presente informe documenta el análisis, la implementación y la verificación del microservicio `ms-audit`, componente responsable de la trazabilidad y auditoría dentro del sistema distribuido *Parqueadero Inteligente*, una plataforma de gestión de estacionamientos construida bajo una arquitectura de microservicios. `ms-audit` centraliza el registro de las operaciones críticas (creación, actualización, eliminación e inicio de sesión) que ocurren en los demás microservicios del sistema (`personas`, `vehiculos`, `zonas`, `ms-tickets` y `asignacion-trazabilidad`), recibéndolas de forma asíncrona a través del bróker de mensajería RabbitMQ, sin acoplarse directamente a ninguno de ellos. Se describe su arquitectura interna basada en NestJS y TypeORM, el patrón productor-consumidor implementado mediante un exchange de tipo *topic*, el esquema de validación estricto de los eventos recibidos, y el mecanismo de seguridad basado en JSON Web Tokens (JWT) y control de acceso por roles que restringe la consulta de los registros de auditoría exclusivamente al personal administrativo. Como parte del desarrollo práctico se identificó y corrigió una falla de autorización en la que el guardián de roles no evaluaba correctamente los metadatos declarados a nivel de clase, permitiendo que un usuario sin privilegios accediera a información sensible; su corrección y verificación se documentan como parte de los resultados obtenidos.

1 Introducción

Los sistemas distribuidos contruidos bajo el paradigma de microservicios dividen la lógica de negocio en unidades pequeñas, independientes y desplegables por separado, cada una con su propia base de datos y ciclo de vida. Esta independencia, si bien favorece la escalabilidad y el mantenimiento del software, introduce un problema recurrente: la dificultad de reconstruir, después de ocurrido un incidente, una línea de tiempo coherente de *qué* sucedió, *quién* lo hizo, *desde dónde* y *cuándo*, cuando la información relevante está dispersa entre varios servicios que no comparten una base de datos común.

El proyecto *Parqueadero Inteligente*, desarrollado como práctica del curso de Aplicaciones Distribuidas, implementa un sistema de gestión de estacionamientos compuesto por seis microservicios (`personas`, `vehiculos`, `zonas`, `ms-tickets`, `asignacion-trazabilidad` y `ms-audit`), un API Gateway (Kong) que centraliza el enrutamiento hacia ellos, y un bróker de mensajería (RabbitMQ) que habilita comunicación asíncrona entre servicios. Ante la necesidad de contar con un registro de auditoría fiable, íntegro y desacoplado del resto del sistema, se diseñó `ms-audit` como el único responsable de persistir el historial de acciones relevantes ocurridas en cualquier otro microservicio, recibéndolas mediante mensajes publicados en una cola de RabbitMQ en lugar de exponer una API que cada servicio deba invocar de forma síncrona.

Este informe describe en detalle la arquitectura, el flujo de datos y las decisiones de diseño de `ms-audit`, así como el trabajo de aseguramiento realizado sobre sus endpoints HTTP de consulta, dado que la información de auditoría – nombres de usuario, direcciones IP, direcciones MAC y el detalle completo de cada operación– constituye en sí misma un activo sensible que requiere protección.

2 Objetivos

Objetivo General

Analizar, implementar y verificar el microservicio `ms-audit` como componente de trazabilidad del sistema distribuido *Parqueadero Inteligente*, describiendo su arquitectura orientada a eventos, su mecanismo de comunicación asíncrona mediante RabbitMQ y el esquema de seguridad que protege el acceso a la información de auditoría.

Objetivos Específicos

1. Describir la arquitectura interna de **ms-audit** (módulos, entidad de persistencia, esquema de validación de datos y consumidor de RabbitMQ), así como el patrón productor-consumidor que emplea para comunicarse de forma desacoplada con el resto de los microservicios del sistema.
2. Implementar y analizar el mecanismo de autenticación basado en JSON Web Tokens y de autorización basada en roles que restringe el acceso a los endpoints HTTP de auditoría exclusivamente a los usuarios con rol **admin** o **root**, dado el carácter sensible de la información que gestiona el servicio.
3. Verificar, mediante pruebas funcionales con peticiones HTTP autenticadas y no autenticadas, el correcto funcionamiento del microservicio dentro del flujo de eventos del sistema y la efectividad de los controles de seguridad implementados.

3 Desarrollo

3.1 Arquitectura general del sistema

El sistema *Parquadero Inteligente* sigue una arquitectura de microservicios desplegada mediante contenedores Docker orquestados con Docker Compose. Cada microservicio posee su propia base de datos PostgreSQL (aunque todas residen en una misma instancia del motor de base de datos, cada una en un esquema independiente: `personas_db`, `vehiculos_db`, `zonas_db`, `tickets_db`, `asignacion_db` y `audit_db`), y todo el tráfico externo hacia los servicios pasa a través de Kong, el cual centraliza políticas transversales como CORS y limitación de tasa (*rate limiting*).

La comunicación entre microservicios ocurre de dos formas: síncrona, mediante peticiones HTTP directas (por ejemplo, cuando `ms-tickets` consulta el estado de un espacio en `zonas`), y asíncrona, mediante RabbitMQ, empleada exclusivamente para el flujo de auditoría. `ms-audit` es, en este sentido, un *consumidor puro*: no expone ninguna funcionalidad de negocio, y su única responsabilidad es recibir, validar y persistir los eventos que los demás servicios publican.

3.2 Patrón productor-consumidor con RabbitMQ

Cada uno de los microservicios que modifica datos de negocio (`personas`, `vehiculos`, `zonas`, `ms-tickets`, `asignacion-trazabilidad`) incorpora una clase `EventPublisher` que, tras cada operación relevante (creación, actualización, eliminación o inicio de sesión), construye un objeto `AuditEvent` y lo publica en un *exchange* de tipo `topic` llamado `audit_exchange`, usando la clave de enrutamiento `audit.event`. El publicador declara el *exchange* como durable pero no declara ninguna cola: esa responsabilidad recae exclusivamente en el consumidor.

```
1 async publish(event: AuditEvent): Promise<void> {
2   const mac = event.mac ?? this.getLocalMacAddress();
3   await this.channel.assertExchange(this.exchange, 'topic', { durable: true });
4   this.channel.publish(
5     this.exchange,
6     this.routingKey,
7     Buffer.from(JSON.stringify({ ...event, mac })),
8     { persistent: true },
9   );
10 }
```

Listing 1: Publicación de un evento de auditoría (`event-publisher.service.ts`, común a los microservicios productores)

Del lado de `ms-audit`, la clase `AuditConsumer` implementa los ganchos de ciclo de vida `OnModuleInit` y `OnModuleDestroy` de NestJS. Al iniciar, declara el mismo `exchange`, declara una cola durable `audit_queue` y la enlaza (`bind`) al `exchange` mediante la clave `audit.event`. Por cada mensaje recibido, el contenido JSON se transforma a una instancia de `CreateAuditEventDto` y se valida con `class-validator`; si la validación falla, el mensaje se rechaza de forma permanente (`nack` sin reencolar) para evitar ciclos infinitos de reintento; si es válido, se persiste mediante `AuditService` y se confirma (`ack`).

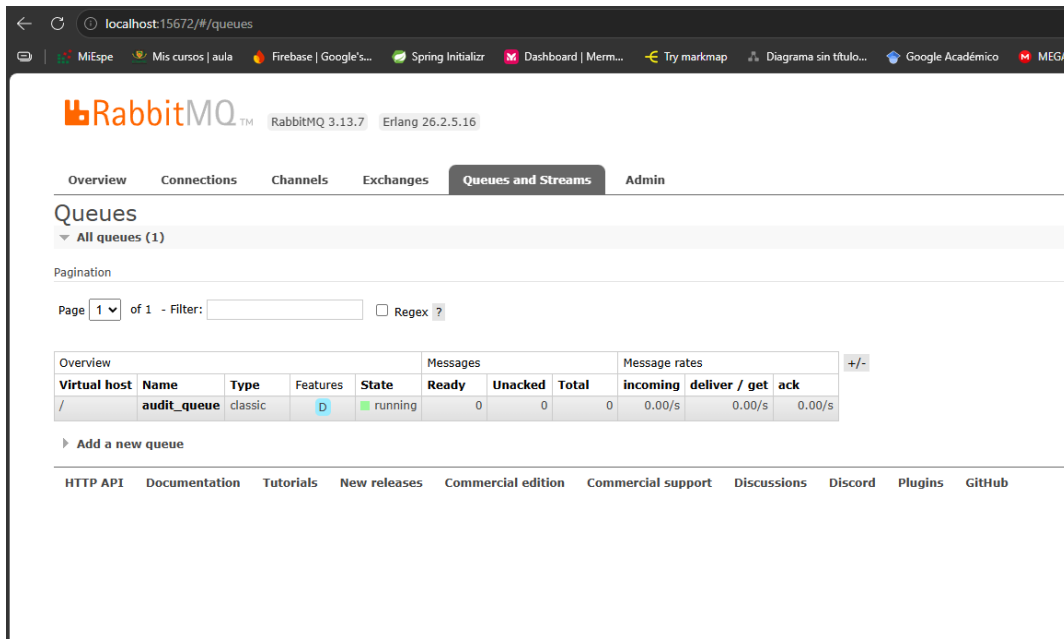


Figura 1: Panel de administración de RabbitMQ mostrando la cola `audit_queue`, de tipo `classic` y durable (columna `Features`: D), en estado `running`.

Como se observa en la Figura 1, la cola `audit_queue` queda registrada y operativa en el bróker una vez que `ms-audit` se inicia, lista para recibir los mensajes enlazados desde `audit_exchange`.

```

1 const evento = plainToInstance(CreateAuditEventDto, JSON.parse(msg.content.toString()));
2 const errores = await validate(evento);
3
4 if (errores.length > 0) {
5   this.channel.nack(msg, false, false); // descarta, no reencola
6   return;
7 }
8
9 await this.auditService.create(evento);
10 this.channel.ack(msg);

```

Listing 2: Validación y persistencia de un mensaje entrante (`audit.consumer.ts`)

Adicionalmente, tanto el publicador como el consumidor implementan una estrategia de reconexión automática: si la conexión con RabbitMQ se pierde o falla al establecerse, se programa un nuevo intento cada 5 segundos, de modo que una caída temporal del bróker no requiera reiniciar manualmente los microservicios involucrados.

3.3 Esquema de datos y validación

La entidad `EventoAuditoria`, mapeada a la tabla `eventos_auditoria` mediante `TypeORM`, almacena: un identificador `UUID`, el nombre del servicio de origen (`servicio`), la acción realizada (`action`), la entidad de negocio afectada (`entidad`), un campo `JSON` con el detalle de los datos (`datos`), el nombre de usuario y rol de quien ejecutó la acción (cuando la petición vino autenticada), la dirección `IP`, la dirección `MAC`, y una marca de tiempo generada automáticamente.

El DTO de entrada (`CreateAuditEventDto`) aplica validaciones estrictas mediante expresiones regulares y enumeraciones cerradas: el campo `servicio` debe iniciar con el prefijo `ms-`; `accion` debe pertenecer al conjunto `{CREATE, UPDATE, DELETE, LOGIN, LOGOUT, SELECT}`; `entidad` solo admite letras mayúsculas y guiones; la dirección `IP` debe ser una `IPv4` válida (`@IsIP('4')`) y la dirección `MAC` debe superar la validación de `@IsMACAddress`. Este nivel de validación evita que datos malformados o incompletos lleguen a persistirse, actuando como una segunda barrera de calidad además de la que ya aplican los microservicios productores sobre sus propios datos de negocio.

3.4 Seguridad: autenticación y autorización

A diferencia de otros microservicios del sistema – que exponen algunos endpoints de lectura pública, por ejemplo la consulta de espacios disponibles para el dashboard del cliente–, `ms-audit` no posee ninguna ruta abierta. Esta decisión de diseño responde a que los registros de auditoría exponen información potencialmente sensible (identidad de los usuarios, direcciones `IP` y `MAC`, y el detalle completo de cada operación realizada en el sistema), por lo que su acceso debe limitarse estrictamente al personal con funciones administrativas.

El controlador `AuditController` aplica, a nivel de toda la clase, un guardián de autenticación (`JwtAuthGuard`) que exige un token `JWT` válido en el encabezado `Authorization`, y un guardián de autorización (`RolesGuard`) que exige que el token contenga el rol `admin` o `root`:

```
1 const AUDIT_ROLES = ['admin', 'root'];
2
3 @Controller('audit')
```



```
4 @UseGuards(JwtAuthGuard, RolesGuard)
5 @Roles(...AUDIT_ROLES)
6 export class AuditController {
7   @Post() create(...) { ... }
8   @Get() findAll(...) { ... }
9   @Get(':id') findOne(...) { ... }
10 }
```

Listing 3: Restricción de acceso a nivel de clase (audit.controller.ts)

Durante la fase de verificación se detectó que, si bien el decorador `@Roles()` estaba correctamente declarado a nivel de clase, el guardián `RolesGuard` obtenía los metadatos de roles requeridos únicamente a través de `reflector.get('roles', context.getHandler())`, método que solo inspecciona el *método* HTTP invocado y no la *clase* que lo contiene. Como consecuencia, el guardián nunca encontraba la restricción declarada y se comportaba como si ningún rol fuera requerido, permitiendo que cualquier usuario autenticado – sin importar su rol– accediera a los registros de auditoría. La corrección consistió en emplear `reflector.getAllAndOverride`, que revisa primero el método y, de no encontrar metadatos ahí, recurre a la clase:

```
1 const requiredRoles = this.reflector.getAllAndOverride<string[]>('roles', [
2   context.getHandler(),
3   context.getClass(),
4 ]);
```

Listing 4: Corrección del guardián de roles (roles.guard.ts)

Esta corrección se aplicó de forma preventiva también en los guardianes equivalentes de los demás microservicios, aun cuando en ellos el decorador se usaba correctamente a nivel de método, con el fin de blindar el sistema completo frente a la misma clase de error si en el futuro se declarara un `@Roles()` a nivel de clase en cualquiera de ellos.

3.5 Configuración de despliegue y enrutamiento

`ms-audit` se despliega como un contenedor independiente (`ms-audit-app`) escuchando en el puerto 3004, con las variables de entorno necesarias para conectarse a su base de datos PostgreSQL (`audit_db`) y al bróker RabbitMQ, además de un secreto compartido de JWT (`JWT_SECRET`) idéntico al utilizado por el microservicio `personas`, que es quien emite los tokens al iniciar sesión. El API Gateway Kong registra una ruta `/audit` hacia este servicio, con la política de CORS habilitada para permitir su consumo desde el dashboard web, quedando accesible externamente en `http://localhost:8000/audit`.

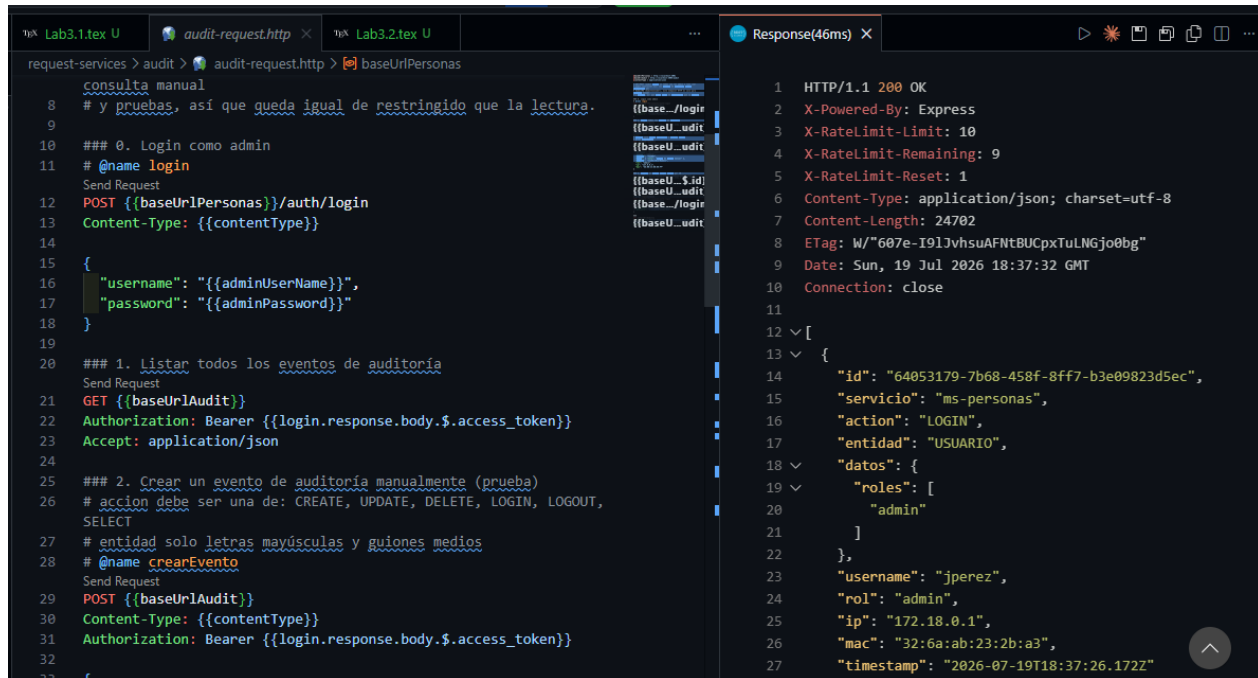
3.6 Verificación funcional

Se realizaron las siguientes pruebas para verificar el correcto funcionamiento del microservicio y de sus controles de seguridad:

- **Flujo asíncrono automático:** al ejecutar operaciones de creación, actualización o inicio de sesión en cualquiera de los microservicios productores, se verificó que el evento correspondiente aparecía persistido en `ms-audit` sin intervención manual, confirmando el correcto funcionamiento del canal RabbitMQ.
- **Acceso sin token:** una petición `GET /audit` sin encabezado `Authorization` devolvió `401 Unauthorized`.
- **Acceso con rol insuficiente:** una petición `GET /audit` con un token válido de un usuario con rol `cliente` devolvió `403 Forbidden`, confirmando la corrección del guardián de roles descrita en la sección anterior.
- **Acceso autorizado:** la misma petición, empleando un token de un usuario con rol `admin`, devolvió `200 OK` junto con el listado completo de eventos registrados.

La Figura 2 evidencia, además de la respuesta autorizada, dos aspectos adicionales de la implementación: la limitación de tasa configurada mediante `THROTTLE_TTL` y `THROTTLE_LIMIT` (visible en los encabezados `X-RateLimit-Limit` y `X-RateLimit-Remaining`), y la estructura real de un evento persistido, con su identificador `UUID`, el servicio de origen, la acción registrada y los metadatos de identidad (usuario, rol, IP y MAC) capturados automáticamente por el sistema.

Estas pruebas confirman que `ms-audit` cumple su doble propósito: registrar de forma confiable y desacoplada toda actividad relevante del sistema, y restringir el acceso a esa información únicamente al personal autorizado.



```
request-services > audit > audit-request.http > baseUrlPersonas
consulta manual
# y pruebas, así que queda igual de restringido que la lectura.
## 0. Login como admin
# @name login
Send Request
POST {{baseUrlPersonas}}/auth/login
Content-Type: {{contentType}}

{
  "username": "{{adminUserName}}",
  "password": "{{adminPassword}}"
}

## 1. listan todos los eventos de auditoría
Send Request
GET {{baseUrlAudit}}
Authorization: Bearer {{login.response.body$.access_token}}
Accept: application/json

## 2. Crear un evento de auditoría manualmente (prueba)
# acción debe ser una de: CREATE, UPDATE, DELETE, LOGIN, LOGOUT,
# entidad solo letras mayúsculas y guiones medios
# @name crearEvento
Send Request
POST {{baseUrlAudit}}
Content-Type: {{contentType}}
Authorization: Bearer {{login.response.body$.access_token}}
```

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 X-RateLimit-Limit: 10
4 X-RateLimit-Remaining: 9
5 X-RateLimit-Reset: 1
6 Content-Type: application/json; charset=utf-8
7 Content-Length: 24702
8 ETag: W/"607e-I9lJvhsuAFntBUCpxTuLNGjo0bg"
9 Date: Sun, 19 Jul 2026 18:37:32 GMT
10 Connection: close
11
12 [
13   {
14     "id": "64053179-7b68-458f-8ff7-b3e09823d5ec",
15     "servicio": "ms-personas",
16     "action": "LOGIN",
17     "entidad": "USUARIO",
18     "datos": {
19       "roles": [
20         "admin"
21       ],
22       "username": "jperez",
23       "rol": "admin",
24       "ip": "172.18.0.1",
25       "mac": "32:6a:ab:23:2b:a3",
26       "timestamp": "2026-07-19T18:37:26.172Z"
27     }
28   }
29 ]
```

Figura 2: Ejecución del flujo de `audit-request.http` (login como administrador seguido de `GET /audit`) desde el cliente REST de Visual Studio Code. La respuesta muestra el código 200 OK, los encabezados de limitación de tasa (`X-RateLimit-*`) aplicados por `ThrottlerModule`, y el primer evento de auditoría del listado, correspondiente a un inicio de sesión.

4 Conclusiones

- La arquitectura orientada a eventos, implementada mediante un exchange de tipo *topic* en RabbitMQ, permite que `ms-audit` registre la actividad de los demás microservicios sin que estos conozcan su existencia ni dependan de su disponibilidad, logrando un desacoplamiento real entre los productores de eventos y su consumidor.
- La estrategia de reconexión automática implementada tanto en el publicador como en el consumidor otorga resiliencia al sistema frente a caídas temporales del bróker de mensajería, evitando la pérdida de eventos y la necesidad de reinicios manuales.
- La validación estricta de los eventos entrantes mediante `class-validator`, junto con el rechazo permanente de mensajes malformados, protege la integridad de los datos de auditoría frente a productores mal configurados.
- La protección de los endpoints de auditoría mediante autenticación JWT y autorización basada en roles es indispensable dado el carácter sensible de la información gestiona-

da; su ausencia habría expuesto identidades, direcciones IP y el historial completo de operaciones del sistema a cualquier usuario autenticado.

- El hallazgo y la corrección del error en la evaluación de metadatos a nivel de clase en **RolesGuard** evidencian la importancia de verificar de forma explícita, mediante pruebas funcionales reales con distintos roles, que los controles de seguridad declarados en el código efectivamente se apliquen en tiempo de ejecución.

5 Recomendaciones

- Incorporar pruebas automatizadas de integración que ejecuten, en cada cambio de código, los mismos escenarios de autorización verificados manualmente (sin token, con rol insuficiente, con rol autorizado), de modo que una regresión como la detectada en **RolesGuard** sea detectada antes de llegar a un entorno productivo.
- Implementar mecanismos de monitoreo sobre la profundidad de la cola `audit_queue` y sobre la tasa de mensajes rechazados (`nack`), que permitan detectar tempranamente productores mal configurados o una eventual saturación del consumidor.
- Definir una política de retención y posible archivado en frío de los registros de auditoría, dado que se trata de una tabla de solo inserción cuyo crecimiento es indefinido en el tiempo.
- Evaluar la incorporación de mecanismos de integridad (por ejemplo, encadenamiento criptográfico de eventos o firmas digitales) que permitan detectar si un registro de auditoría fue alterado después de su creación, elevando el nivel de garantía que ofrece el sistema ante una auditoría externa.
- Añadir paginación a la consulta `GET /audit`, actualmente diseñada para devolver la totalidad de los registros, con el fin de mantener un rendimiento aceptable a medida que el volumen histórico de eventos crezca.

Referencias

- [1] NestJS. (2024). *NestJS – A progressive Node.js framework*. Documentación oficial. <https://docs.nestjs.com>
- [2] RabbitMQ. (2024). *RabbitMQ tutorials – Work queues, topics and reliability*. Documentación oficial de VMware/Broadcom. <https://www.rabbitmq.com/tutorials>
- [3] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2.^a ed.). O'Reilly Media.
- [4] OWASP Foundation. (2023). *Logging Cheat Sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- [5] Jones, M., Bradley, J., y Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc7519>